Dominando Testes em CAP Node.js

Um Guia Visual para a Biblioteca `@cap-js/cds-test`

Transforme seus testes de um dever para uma vantagem estratégica.
Aprenda a escrever testes robustos, portáteis e eficientes.

NotebookLM

Por que `@cap-js/cds-test`? A Abstração que Você Precisava.

Simplicidade

Inicia e encerra um servidor CAP automaticamente dentro dos seus testes (`beforeAll` / `afterAll`), eliminando o boilerplate.

Consistência

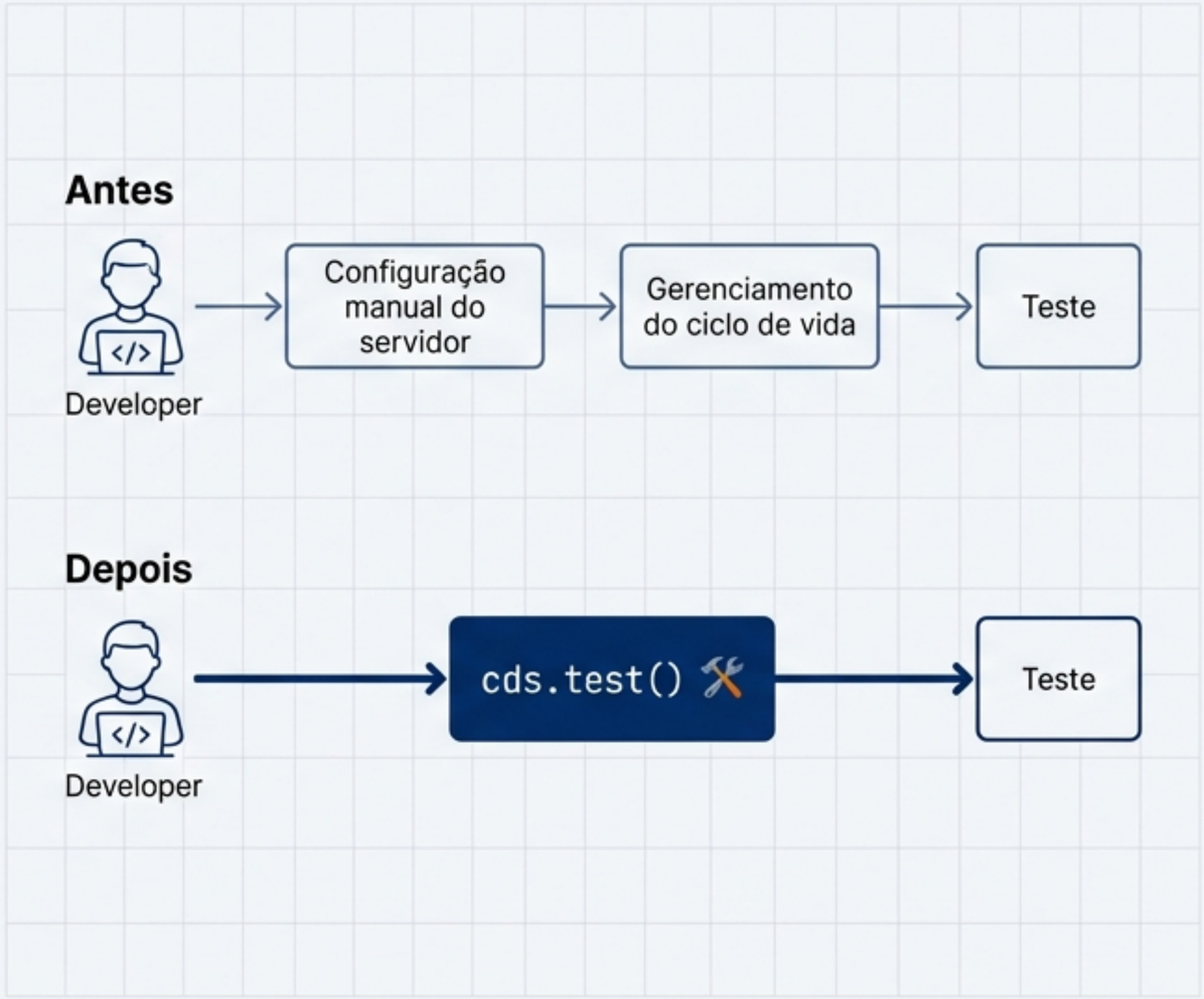
Fornece utilitários de melhores práticas para testar APIs de serviço (programáticas) e APIs HTTP.

Portabilidade

Projetada para escrever testes que rodam tanto em Jest quanto em Mocha sem alterações.

Ferramentas Integradas

Vem com um cliente HTTP (Axios) pré-configurado e a biblioteca de asserções Chai pronta para uso.



NotebookLM

Seu Primeiro Teste em Minutos.

Passo 1: Instalação

```
npm add -D @cap-js/cds-test
```

Passo 2: Estrutura do Projeto



Passo 3: Código do Teste

Em seu arquivo `.test.js`, inicie o servidor com uma única linha. Ele será gerenciado automaticamente.

```
const cds = require('@sap/cds')

describe('Meu primeiro conjunto de tes
  const test = cds.test(__dirname + '/.

  it('deve funcionar', () => {
    // Seus testes aqui...
    expect(true).to.be.true
  })
})
```

NotebookLM

Acessando Serviços Programaticamente.

Conceito

Como o servidor roda no mesmo processo, é possível conectar-se diretamente aos serviços usando la API Node.js do CAP. Isso é ideal para testar a lógica do serviço sem a sobrecarga do HTTP. `cds.connect.to('ServiceName')` é a porta de entrada.

Exemplo de Código

```
it('Permite testar APIs programáticas', async () => {
  const AdminService = await cds.connect.to('AdminService')
  const { Authors } = AdminService.entities

  // Três formas de obter o mesmo resultado
  expect(await SELECT.from(Authors))
    .to.eql(await AdminService.read(Authors))
    .to.eql(await AdminService.run(SELECT.from(Authors)))
})
```

NotebookLM

Testando Endpoints HTTP com Facilidade 

Conceito

- A **biblioteca** fornece **funções de atalho** (`GET`, `POST`, `PUT`, etc.) vinculadas à instância de teste.
- Essas funções usam um cliente Axios pré-configurado com a URL base do servidor, **então você só precisa fornecer os caminhos relativos.**

Exemplo de Código

```
const { GET, POST } = cds.test(...) // Desestruturação do
objeto de teste

it('deve buscar livros e submeter um pedido', async () => {
  // Buscando uma lista de livros
  const { data } = await GET('/browse/Books')
  expect(data.value).to.not.be.empty

  // Enviando um novo pedido
  const response = await POST('/browse/submitOrder', {
    book: 201, quantity: 5 })
  expect(response.status).to.equal(201) // Exemplo
})
```

NotebookLM



Lidando com Endpoints Autenticados.

Conceito

- Por padrão, `cds.test` usa a estratégia de autenticação mockada do CAP.
- Você pode facilmente personificar usuários em suas requisições HTTP passando um objeto `auth`.
- Isso é equivalente a configurar o cabeçalho `Authorization: Basic ...` manualmente, mas de forma muito mais limpa.
- Menciona que usuários mockados podem ser configurados no `package.json`.

Exemplo de Código e Visual

```
// Requisição como a usuária 'alice'
await GET('/admin/Books', {
  auth: { username: 'alice', password: '' }
})
```

O que acontece por baixo dos panos:

```
/*
GET http://localhost:4004/admin/Books
Authorization: Basic alice:
*/
```

NotebookLM



Escreva uma Vez, Rode em Qualquer Lugar: Jest & Mocha.

Conceito

- **A Chave para a Portabilidade:** Use o `expect` fornecido pela biblioteca, que é uma instância do Chai. A sintaxe do Chai é poderosa e funciona universalmente.
- A estrutura (`describe`, `it`, `test`) é comum a ambos.

> ⚠ **Aviso**
>
> Evite helpers específicos do Jest como `jest.resetModules` e `jest.useFakeTimers`, pois podem causar estados inconsistentes no servidor.

Exemplo de Código

```
describe('my portable test suite', () => {
  const { GET, expect } = cds.test(...) // Importante ob

  it('should test data', async () => { // Funciona em Je
    const { data } = await GET('/browse/Books')

    expect(data.value).to.eql([ // Estilo de asserção do
      { ID: 201, title: 'Wuthering Heights', /*...*/ },
      //...
    ])
  })
})
```

NotebookLM

Seu Kit de Ferramentas: Explorando o Objeto `test`.



`test.expect`

Atalho para `chai.expect()`. A maneira recomendada para asserções portáteis.

```
expect(foo).to.have.property('bar')
```



`test.axios`

Acesso direto à instância Axios pré-configuada com a URL base. **Use para cenários HTTP avançados.**

```
const res = await test.axios.get('/catalog/Books')
```



`GET`, `POST`, ... (atalhos)

Funções `GET`, `POST`, etc., vinculadas, para requisições HTTP concisas. Podem ser usadas como tagged templates.

```
const { data } = await GET `/browse/Books`
```



test.chai

Acesso à biblioteca Chai completa, pré-configurada com plugins úteis como `chai-subset` e `chai-as-promised`.

```
expect(obj).to.containSubset({...})
```

NotebookLM

Garantindo a Consistência: Resetando o Banco de Dados

Conceito

- Testes devem ser independentes e não devem ser afetados pelo estado deixado por testes anteriores.
- O método `test.data.reset()` limpa e reimplanta o banco de dados (em memória, por padrão) com os dados iniciais.
- O uso mais comum é em um hook `beforeEach` para garantir que cada teste (`it`) comece do zero.

Exemplo de Código

Forma mais limpa e comum:

```
const { test } = cds.test()
describe('conjunto de testes isolados', () => {
  beforeEach(test.data.reset)
  it('teste 1', async () => { /* ... */ })
  })
  it('teste 2', async () => { /* ... */ })
})
```

Forma alternativa explícita:

```
beforeEach(async () => {
  await test.data.reset()
})
```

NotebookLM

Depuração Eficiente: Capturando Logs do Console.



Conceito

- O método `test.log()` captura toda a saída de `console.log`, `console.error`, etc., dentro de um escopo de teste (`describe`).
- Retorna um objeto com a saída (`.output`) e métodos para controlar a captura (`.clear()`, `.release()`).
- A biblioteca gerencia a captura e limpeza automaticamente entre os testes.

Exemplo de Código

```
describe('testando a captura de logs', () => {
  let log = cds.test.log() // Inicia a captura para

  it('deve capturar a saída do log', () => {
    console.log('foo', { bar: 2 })
    expect(log.output).to.contain('foo')
  })

  it('deve limpar o log entre os testes', () => {
    // log.output já está vazio aqui
    expect(log.output).to.equal('')
  })
})
```

NotebookLM



Controle Avançado do Servidor e do Contexto

Conceito

- **test.run(...)**: O método por trás de **cds.test()**. Permite passar argumentos como na linha de comando **cds serve**. Útil para servir apenas serviços específicos.
- **test.in(folder)**: O método mais importante para controle de contexto. Muda o **cds.root** para a pasta especificada, garantindo que o modelo, as configurações e os dados corretos sejam carregados.

Exemplos de Código

```
// Padrão mais comum: servir tudo a partir de uma pasta específica
cds.test().in('/cap/samples/bookshop')

// Servir apenas um serviço específico
cds.test('serve', 'CatalogService').in('/cap/samples/bookshop')

//  Mudar o diretório raiz sem iniciar um servidor
cds.test.in(__dirname)
```

💡 **Dica Pro**

Use **test.in(folder)** de forma fluida para combinar inicialização e mudança de contexto em uma única linha.

NotebookLM

A Armadilha do `cds.env`: A Ordem de Execução Importa.

O Problema

Se `cds.env` ou qualquer módulo `cds` (como `cds.Service`) for acessado *antes* de `cds.test(folder)` ou `cds.test.in(folder)`, as configurações podem ser carregadas do diretório errado, causando erros difíceis de depurar.

```
// ERRADO!
const cds = require('@sap/cds')
class MyService extends cds.Service {} // cds.env é carregado aqui do ./

describe('suite', () => {
  cds.test(__dirname) // Tenta mudar o contexto tarde demais
})
```

A Solução

Sempre chame `cds.test(folder)` ou `cds.test.in(folder)` como a ***primeira instrução*** no seu arquivo de teste, antes de qualquer outra referência ao `cds`.

```
const cds = require('@sap/cds')

describe('suite', () => {
  cds.test(__dirname) // CORRETO! Primeira coisa dentro do describe

  class MyService extends cds.Service {} // Agora carrega do contexto c
})
```

Dica Pro: Habilite a Verificação

Execute seus testes com a variável de ambiente `CDS_TEST_ENV_CHECK=y` para detectar esses erros automaticamente. É provável que se torne o padrão no futuro.

```
CDS_TEST_ENV_CHECK=y jest
```

NotebookLM

Escrevendo Testes Resilientes: Melhores Práticas.



1. Verifique o Status por Último (se necessário)

Não aborte o teste com uma verificação de status. Verifique os dados primeiro para obter mensagens de erro mais ricas.

❌ **NÃO**

`expect(status).to.equal(200)` no início

✅ **FAÇA**

`expect(data).to.equal(...)` primeiro. A verificação do status é opcional.



2. Evite Suposições Mínimas sobre Erros

Não dependa de mensagens de erro exatas, que podem mudar. Verifique apenas a palavra-chave essencial do erro.

❌ **NÃO**

```
rejectedWith('Entity
"CatalogService.Books" is readonly')
```

✅ **FAÇA**

```
rejectedWith(/readonly/i)
```



3. Mantenha o Código de Teste Agnóstico ao Ambiente

A configuração do ambiente (bindings, perfis) deve ser feita em scripts de CI/CD, não no código do teste. Isso mantém seus testes isolados e reprodutíveis.

NotebookLM



Maestria Interativa: Usando `cds.test` no REPL

Conceito

- Você pode usar `cds.test` diretamente no REPL (`cds repl`) para iniciar um servidor de teste interativamente.
- Isso permite que você execute queries, chame serviços e inspecione dados em tempo real sem escrever um arquivo de teste completo.
- Uma ferramenta poderosa para depuração rápida e exploração do seu modelo e serviços.

```
[cap/samples] cds repl
> var test = await cds.test('bookshop')
[cds] - server listening on { url: 'http://localhost:64914' }

...

> await SELECT `title` .from `Books` .where `exists
author[name like '%Poe%']`
[ { title: 'The Raven' }, { title: 'Eleonora' } ]

> var { CatalogService } = cds.services
> await CatalogService.read `title, author` .from
`ListOfBooks`
[ { title: 'Wuthering Heights', author: 'Emily Brontë' }, … ]
```

NotebookLM